
NAME: S.DHARSHINI

REG NO: 192424258

SUB CODE: DSA0502

SUB NAME: QUERY PROCESSING FOR DATA SCIENCE

CO2 ASSESSMENT TOOL 2

04.08.2026

3. Online Shopping System

Database tables include:

Customer(CustomerID, Name)

Product(ProductID, ProductName, Price)

Order(OrderID, CustomerID, OrderDate)

OrderDetails(OrderID, ProductID, Quantity)

Each order may contain multiple products.

Viva Questions

Q1. Why is the OrderDetails table required in this schema?

Q2. What would happen if ProductID were stored directly in the Order table?

AIM: To develop an Online Shopping System using Python and MySQL to manage customers, products, orders, and order details efficiently.

PROCEDURE:

- Create the ShoppingDB database.
- Create Customer, Product, Orders, and OrderDetails tables.
- Insert sample customer and product records.
- Connect Python with MySQL.

- Register a customer.
- Place an order.
- Store ordered products in the OrderDetails table.
- Display order details using SQL JOIN.
- Verify the output.

OUTPUT:

The first screenshot shows the MySQL 8.0 Command Line Client with the following commands and output:

```
mysql> CREATE DATABASE ShoppingDB;
Query OK, 1 row affected (0.002 sec)

mysql> USE ShoppingDB;
Database changed

mysql> CREATE TABLE Customer(
  > CustomerID INT PRIMARY KEY AUTO_INCREMENT,
  > Name VARCHAR(100)
  > );
Query OK, 0 rows affected (0.002 sec)

mysql> CREATE TABLE Product(
  > ProductID INT PRIMARY KEY AUTO_INCREMENT,
  > ProductName VARCHAR(100),
  > Price DECIMAL(9,2)
  > );
Query OK, 0 rows affected (0.002 sec)

mysql> CREATE TABLE Orders(
  > OrderID INT PRIMARY KEY AUTO_INCREMENT,
  > CustomerID INT,
  > OrderDate DATE,
  > FOREIGN KEY(CustomerID) REFERENCES Customer(CustomerID)
  > );
Query OK, 0 rows affected (0.002 sec)

mysql> desc Orders;
+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+
| OrderID | int | NO | PRI | NULL | auto_increment |
| CustomerID | int | YES | MUL | NULL | |
| OrderDate | date | YES | MUL | NULL | |
+-----+-----+-----+-----+-----+
3 rows in set (0.000 sec)

mysql> CREATE TABLE OrderDetails(
  > OrderID INT,
  > ProductID INT,
  > Quantity INT,
  > PRIMARY KEY(OrderID, ProductID),
  > FOREIGN KEY(OrderID) REFERENCES Orders(OrderID),
  > FOREIGN KEY(ProductID) REFERENCES Product(ProductID)
  > );
Query OK, 0 rows affected (0.002 sec)

mysql> desc OrderDetails;
+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+
| OrderID | int | NO | PRI | NULL | |
| ProductID | int | NO | PRI | NULL | |
| Quantity | int | YES | MUL | NULL | |
+-----+-----+-----+-----+-----+
3 rows in set (0.000 sec)

mysql> SHOW TABLES;
+-----+
| Tables_in_ShoppingDB |
+-----+
| Customer |
| OrderDetails |
| Orders |
| Product |
+-----+
4 rows in set (0.000 sec)

mysql> INSERT INTO Customer(Name)
  > VALUES
  > ('Rahul'),
  > ('Priya');
Query OK, 2 rows affected (0.002 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql> select * from Customer;
+-----+-----+
| CustomerID | Name |
+-----+-----+
| 1 | Rahul |
| 2 | Priya |
+-----+-----+
2 rows in set (0.000 sec)
```

The second screenshot shows the MySQL 8.0 Command Line Client with the following commands and output:

```
mysql> INSERT INTO Product(ProductName, Price)
  > VALUES
  > ('Laptop', 55000),
  > ('Mouse', 700),
  > ('Keyboard', 1200);
Query OK, 3 rows affected (0.002 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> select * from Product;
+-----+-----+-----+
| ProductID | ProductName | Price |
+-----+-----+-----+
| 1 | Laptop | 55000.00 |
| 2 | Mouse | 700.00 |
| 3 | Keyboard | 1200.00 |
+-----+-----+-----+
3 rows in set (0.000 sec)

mysql> SELECT
  > Orders.OrderID,
  > Customer.Name,
  > Product.ProductName,
  > OrderDetails.Quantity,
  > OrderDate
  > FROM Orders
  > JOIN Customer
  > ON Orders.CustomerID = Customer.CustomerID
  > JOIN OrderDetails
  > ON Orders.OrderID = OrderDetails.OrderID
  > JOIN Product
  > ON OrderDetails.ProductID = Product.ProductID;
+-----+-----+-----+-----+-----+
| OrderID | Name | ProductName | Price | Quantity | OrderDate |
+-----+-----+-----+-----+-----+
| 1 | Rahul | Laptop | 55000.00 | 2 | 2026-08-01 |
+-----+-----+-----+-----+-----+
1 row in set (0.001 sec)
```

The screenshot shows the MySQL 8.0 Command Line Client with the following commands and output:

```
mysql> INSERT INTO Product(ProductName, Price)
  > VALUES
  > ('Laptop', 55000),
  > ('Mouse', 700),
  > ('Keyboard', 1200);
Query OK, 3 rows affected (0.002 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> select * from Product;
+-----+-----+-----+
| ProductID | ProductName | Price |
+-----+-----+-----+
| 1 | Laptop | 55000.00 |
| 2 | Mouse | 700.00 |
| 3 | Keyboard | 1200.00 |
+-----+-----+-----+
3 rows in set (0.000 sec)

mysql> SELECT
  > Orders.OrderID,
  > Customer.Name,
  > Product.ProductName,
  > OrderDetails.Quantity,
  > OrderDate
  > FROM Orders
  > JOIN Customer
  > ON Orders.CustomerID = Customer.CustomerID
  > JOIN OrderDetails
  > ON Orders.OrderID = OrderDetails.OrderID
  > JOIN Product
  > ON OrderDetails.ProductID = Product.ProductID;
+-----+-----+-----+-----+-----+
| OrderID | Name | ProductName | Price | Quantity | OrderDate |
+-----+-----+-----+-----+-----+
| 1 | Rahul | Laptop | 55000.00 | 2 | 2026-08-01 |
+-----+-----+-----+-----+-----+
1 row in set (0.001 sec)
```

The screenshot shows a Python IDE with the following code in `display.py`:

```
display.py>
45 print('Order Date : ', row[5])
46 print('-----')
47
48 cursor.close()
49 for i in row:
```

The terminal output shows the following commands and results:

```
D:\Desktop\Online_Shopping_System>python connect.py
Connected Successfully!

D:\Desktop\Online_Shopping_System>python insert_data.py
Enter Customer Name: 1
Customer Added Successfully!

D:\Desktop\Online_Shopping_System>python order.py
Enter Customer ID: 1
Enter Product ID: 1
Enter Quantity: 2
Order Placed Successfully!

D:\Desktop\Online_Shopping_System>python display.py

ONLINE SHOPPING DETAILS

Order ID : 1
Customer : Rahul
Product : Laptop
Price : 55000.00
Quantity : 2
Order Date : 2026-08-01
```

Viva Questions

Q1. Why is the OrderDetails table required in this schema?

The **OrderDetails** table is required because one order can contain multiple products, and one product can appear in many different orders. It stores the **ProductID** and **Quantity** for each product in an order, avoiding data duplication.

Q2. What would happen if ProductID were stored directly in the Order table?

If **ProductID** were stored directly in the **Orders** table, each order could contain only **one product**. To store multiple products, duplicate order records would be required, causing redundancy and making the database inefficient.

CONCLUSION: The Online Shopping System efficiently manages customers, products, and orders using a relational database. The use of the OrderDetails table allows multiple products to be stored for a single order while reducing redundancy.

8. Food Delivery Application

Tables:

Customer(CustomerID, Name)

Restaurant(RestaurantID, Name)

Order(OrderID, CustomerID, RestaurantID)

DeliveryAgent(AgentID, Name)

Delivery(OrderID, AgentID)

Orders are delivered by delivery agents.

Viva Questions

Q1. Why is Delivery maintained as a separate table?

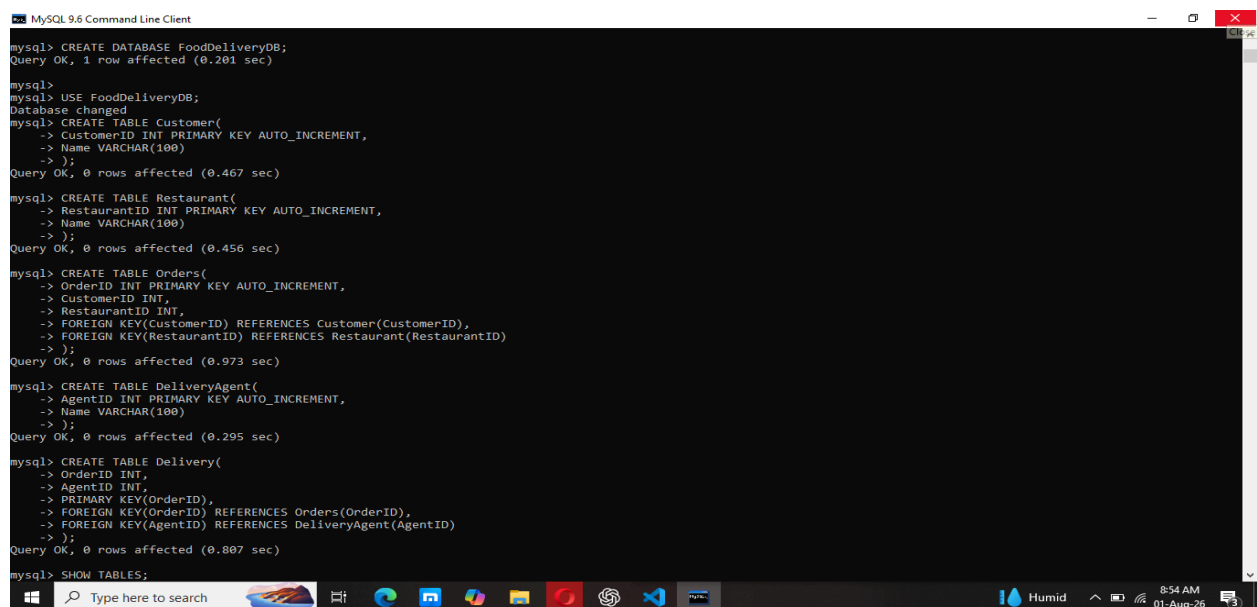
Q2. How would the schema change if multiple delivery agents could handle a single order?

AIM: To develop a Food Delivery Application using Python and MySQL for managing customers, restaurants, food orders, and delivery agents efficiently.

PROCEDURE:

- Create the FoodDeliveryDB database.
- Create Customer, Restaurant, Orders, DeliveryAgent, and Delivery tables.
- Insert sample customer, restaurant, and delivery agent records.
- Connect Python with MySQL.
- Add customer details.
- Place a food order.
- Assign a delivery agent.
- Display food delivery details using SQL JOIN.
- Verify the output.

OUTPUT:



```
mysql> CREATE DATABASE FoodDeliveryDB;
Query OK, 1 row affected (0.201 sec)

mysql>
mysql> USE FoodDeliveryDB;
Database changed
mysql> CREATE TABLE Customer(
  -> CustomerID INT PRIMARY KEY AUTO_INCREMENT,
  -> Name VARCHAR(100)
  -> );
Query OK, 0 rows affected (0.467 sec)

mysql> CREATE TABLE Restaurant(
  -> RestaurantID INT PRIMARY KEY AUTO_INCREMENT,
  -> Name VARCHAR(100)
  -> );
Query OK, 0 rows affected (0.456 sec)

mysql> CREATE TABLE Orders(
  -> OrderID INT PRIMARY KEY AUTO_INCREMENT,
  -> CustomerID INT,
  -> RestaurantID INT,
  -> FOREIGN KEY(CustomerID) REFERENCES Customer(CustomerID),
  -> FOREIGN KEY(RestaurantID) REFERENCES Restaurant(RestaurantID)
  -> );
Query OK, 0 rows affected (0.973 sec)

mysql> CREATE TABLE DeliveryAgent(
  -> AgentID INT PRIMARY KEY AUTO_INCREMENT,
  -> Name VARCHAR(100)
  -> );
Query OK, 0 rows affected (0.295 sec)

mysql> CREATE TABLE Delivery(
  -> OrderID INT,
  -> AgentID INT,
  -> PRIMARY KEY(OrderID),
  -> FOREIGN KEY(OrderID) REFERENCES Orders(OrderID),
  -> FOREIGN KEY(AgentID) REFERENCES DeliveryAgent(AgentID)
  -> );
Query OK, 0 rows affected (0.807 sec)

mysql> SHOW TABLES;
```

```
MySQL 9.6 Command Line Client
mysql> SHOW TABLES;
+-----+
| Tables_in_fooddeliverydb |
+-----+
| customer                  |
| delivery                  |
| deliveryagent              |
| orders                    |
| restaurant                |
+-----+
5 rows in set (0.017 sec)

mysql> INSERT INTO Customer(Name)
-> VALUES
-> ('Rahul'),
-> ('Priya');
Query OK, 2 rows affected (0.136 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql> INSERT INTO Restaurant(Name)
-> VALUES
-> ('A2B Restaurant'),
-> ('Dominos');
Query OK, 2 rows affected (0.095 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql> INSERT INTO DeliveryAgent(Name)
-> VALUES
-> ('Arun'),
-> ('Kumar');
Query OK, 2 rows affected (0.107 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql> select*from Restaurant;
+-----+
| RestaurantID | Name          |
+-----+
| 1            | A2B Restaurant |
| 2            | Dominos       |
+-----+
2 rows in set (0.018 sec)

mysql> SELECT * FROM Customer;
```

```
Select MySQL 9.6 Command Line Client
mysql> SELECT * FROM Customer;
+-----+
| CustomerID | Name |
+-----+
| 1          | Rahul |
| 2          | Priya |
+-----+
2 rows in set (0.005 sec)

mysql> SELECT * FROM DeliveryAgent;
+-----+
| AgentID | Name |
+-----+
| 1        | Arun |
| 2        | Kumar |
+-----+
2 rows in set (0.005 sec)

mysql> SELECT
-> Orders.OrderID,
-> Customer.Name,
-> Restaurant.Name,
-> DeliveryAgent.Name
-> FROM Orders
-> JOIN Customer
-> ON Orders.CustomerID = Customer.CustomerID
-> JOIN Restaurant
-> ON Orders.RestaurantID = Restaurant.RestaurantID
-> JOIN Delivery
-> ON Orders.OrderID = Delivery.OrderID
-> JOIN DeliveryAgent
-> ON Delivery.AgentID = DeliveryAgent.AgentID;
+-----+
| OrderID | Name | Name | Name |
+-----+
| 1       | Rahul | A2B Restaurant | Arun |
+-----+
1 row in set (0.012 sec)
```

```
File Edit Selection View Go Run ... Food_Delivery_System
EXPLORER
FOOD_DELIVERY_SYSTEM
  connect.py
  display.py
  insert_data.py
  order.py
  display.py > ...
    18 FROM Orders
    19
    20 JOIN Customer
    21 ON Orders.CustomerID = Customer.CustomerID
    22
    23 JOIN Restaurant
    24 ON Orders.RestaurantID = Restaurant.RestaurantID
    25
    26 JOIN Delivery
    27
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Connected Successfully!
D:\Desktop\Food_Delivery_System>python insert_data.py
Enter Customer Name: sai
Customer Added Successfully!

D:\Desktop\Food_Delivery_System>python order.py
Enter Customer ID: 1
Enter Restaurant ID: 1
Enter Delivery Agent ID: 1
Order Placed Successfully!

D:\Desktop\Food_Delivery_System>python display.py
FOOD DELIVERY DETAILS
Order ID : 1
Customer : Rahul
Restaurant : A2B Restaurant
Delivery Agent : Arun

Ln 31, Col 44 (344 selected) Spaces: 4 UTF-8 CRLF
Python Python 3.12 8:55 AM 01-Aug-26
```

Viva Questions and Answers

Q1. Why is Delivery maintained as a separate table?

The Delivery table is kept separate because delivery information is different from order information. It stores the relationship between an order and the delivery agent, reducing redundancy and making it easier to manage delivery assignments.

Q2. How would the schema change if multiple delivery agents could handle a single order?

If multiple delivery agents could handle one order, the Delivery table would allow multiple records for the same OrderID. Instead of making OrderID the primary key, a composite primary key (OrderID, AgentID) or a separate DeliveryID column would be used. This would support a many-to-many relationship between orders and delivery agents.

CONCLUSION: The Food Delivery Application efficiently manages food orders and delivery assignments using a relational database. Python and MySQL provide an effective solution for storing, retrieving, and managing food delivery data.